Punteros

Punteros

- *Los punteros en el Lenguaje C, son variables que "apuntan", es decir que poseen la dirección de la ubicación en memoria de otra variable.*
- *Por medio de ellos tenemos un método muy interesante para acceder a la memoria.*
- *Es una de las “especialidades” del lenguaje C.*

Punteros

- *Este tema suele ser bastante conflictivo ya que muchos programadores en otros lenguajes y nuevos en C, lo ven como un método extraño o al menos desacostumbrado, lo que produce cierto rechazo.*
- *Sin embargo, a medida que uno se va familiarizando con ellos, se convierten en una herramienta poderosa.*

Punteros

- *Se usan fundamentalmente para el manejo de variables complejas, argumentos, etc.*
- *También los utilizamos cuando pasamos un parámetro por referencia a una función.*
- *Tienen tipo, igual que las variables comunes, aunque sólo contengan una dirección se debe indicar el tipo.*

Declaración

- *La declaración de un puntero se hace de la siguiente forma:*

```
tipo *nombre;
```

Ejemplo:

```
int *mens;
```

- *mens es un puntero a enteros, debemos incluir siempre el asterisco al declarar punteros.*

Declaración

- *En forma conjunta podemos declarar variables comunes de un tipo y punteros a variables del mismo tipo.*

  Ejemplo:

```
int base, *mens;
```

Declaración

- *La diferencia entre los tipos de la declaración anterior es:*
- `base` *es una variable entera común y el compilador reservará el espacio que ocupa una variable entera (p. ej. 2 bytes).*
- *Para* `mens` *el compilador reserva el espacio necesario para guardar una dirección.*

Aritmética de Punteros

- *Si solo guardan una dirección, ¿porqué hay que asignarles un tipo?*
- *Cuando tenemos una secuencia de variables, pretendemos desplazar un puntero recorriéndolas, habiendo declarado el tipo de variable a que apuntan, el sistema sabe cuántos bytes necesita saltar para llegar a la variable siguiente o a la anterior.*

Aritmética de Punteros

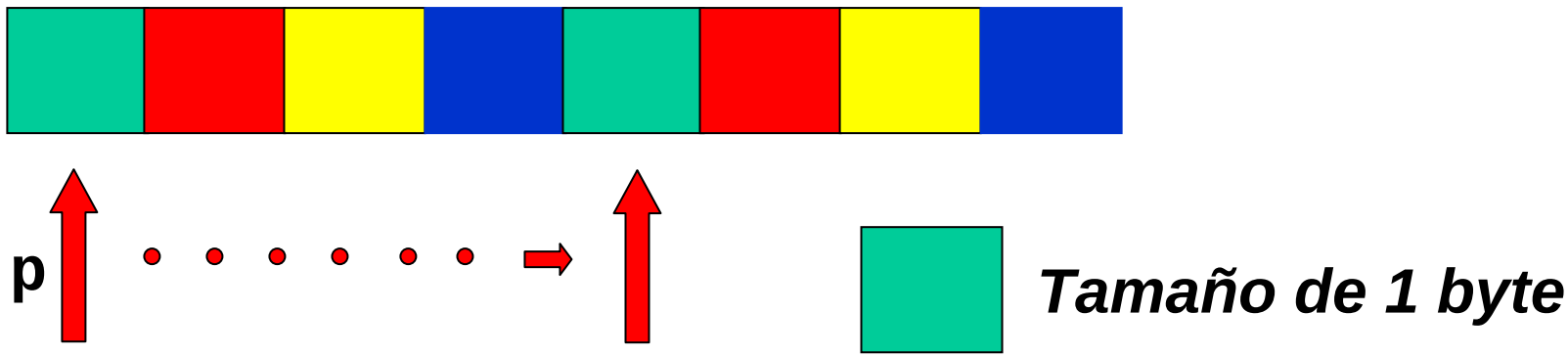
Si teniendo declarado el puntero p como

```
float *p ;
```

Lo movemos un lugar, p. ej.

```
p++ ;
```

El desplazamiento obtenido es el de un elemento del tipo float.



Aritmética de Punteros

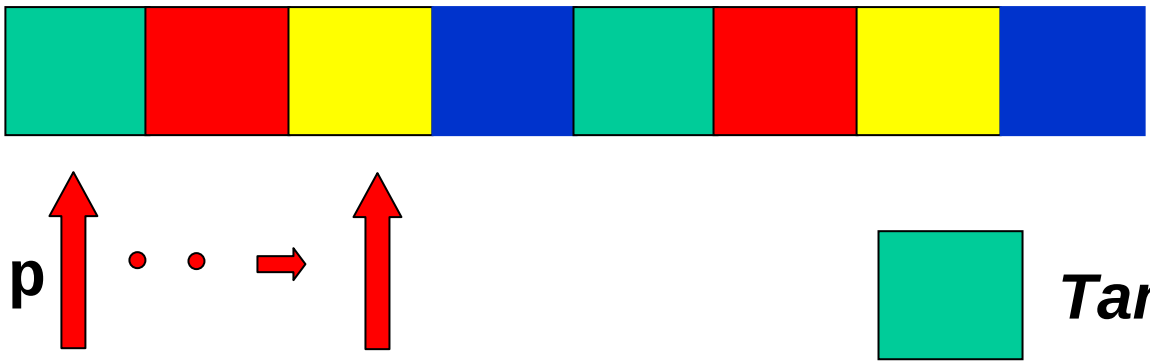
Si en cambio lo declaramos como:

```
int *p ;
```

y lo movemos nuevamente un lugar:

```
p++ ;
```

El desplazamiento obtenido es el de un elemento del tipo int (asumimos 2 bytes).



Aritmética de Punteros

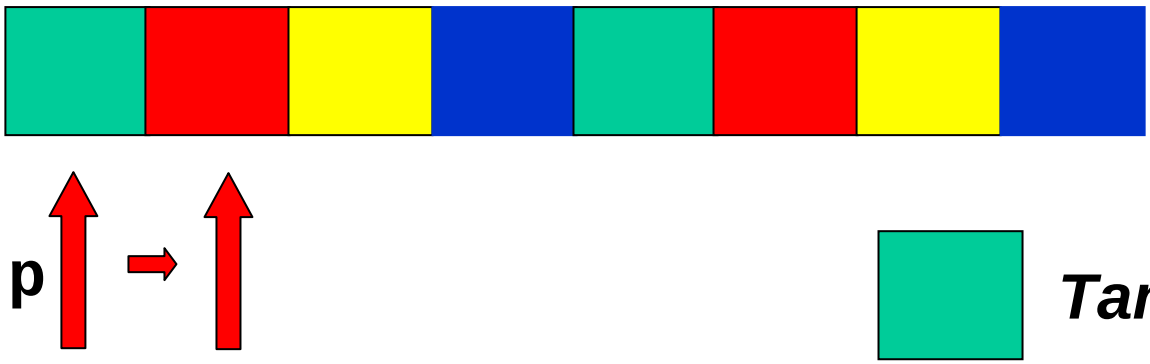
Por último si lo hemos declarado como:

```
char *p ;
```

y lo movemos un lugar:

```
p++ ;
```

El desplazamiento obtenido es el de un elemento del tipo char.



Tamaño de 1 byte

Aritmética de Punteros

- *Esto se denomina:* ***aritmética de punteros****.*
- *Observe que la cantidad de bytes que se desplaza un puntero para saltar de una variable a otra (del mismo tipo) es enteramente dependiente del tamaño que el sistema reserva para ese tipo de variables.*
- *Esto cambia con el compilador.*

Operadores de Punteros

- *Declaramos lo siguiente:*

```
char a, b, *c, *d ;
c = &a ;
*c = 'M' ;
d = c ;
```

- *La primera línea dice que **c** y **d** son punteros a char. La segunda apunta el puntero **c** a la dirección de la variable **a**. (que también es del tipo char)*

Operadores de Punteros

- *En la tercera línea se hace que el contenido del lugar donde apunta **c** sea reemplazado por el código de la letra **M.***
- *En la cuarta línea hacemos que **d** apunte a donde apunta **c**, que es la dirección de la variable **a**.*
- *Puede haber varios punteros apuntando a una misma dirección.*

Operadores de Punteros

- *El operador * sirve declarar un puntero.*
- *También sirve, una vez declarado el puntero para conocer el contenido de las posiciones de memoria apuntadas por el puntero.*
- *El operador & significa:* ***la dirección de.***

Operadores de Punteros

- *Por este motivo en:*

```
c = &a;
```

- *Decimos que **c** apunta a la dirección de **a**.*
- *Esto quiere decir que estamos almacenando la dirección de **a** como contenido de la variable **c**.*
- *Esto es lo que contiene un puntero.*

Operadores de Punteros

- *En:*

```
*c = 'M';
d = c;
```

- *Reemplazamos el contenido de la dirección apuntada por **c**, (lugar de la variable **a**) con el valor ASCII de **M**.*
- *Al final apuntamos **d** donde apunta **c** (copiamos a **d** el contenido de **c**).*

Operaciones con Punteros

- *Sólo dos operaciones son posibles con los punteros, el incremento y el decremento.*

```
p++;
p += 5;
p--:
p -= 5;
```

- *C no ejerce ningún control sobre dónde están apuntando los punteros que se declaran.*

Operaciones con Punteros

- *Un puntero usado sin inicializar es uno de los problemas más graves que podemos tener.*
- *También podemos comparar punteros y verificar si uno apunta a una posición más alta de memoria que otro.*
- *El resultado del punto anterior puede variar con arquitecturas diferentes.*

Derreferenciación de Punteros

- *Llamamos derreferenciar un puntero a la operación de obtener el contenido de la variable a la que apunta.*

```
a = *b ;
```

- *Guardamos en **a** el contenido de las posiciones de memoria donde está apuntando **b**.*

- ***a** no se entera que **b** es un puntero.*

Errores con Punteros

- *Si los punteros no están controlados, producen errores bastante difíciles de detectar.*

  ¿Qué error advierte en este código?

```
int main( void )
{
   int x, *p;
   x = 10;
   *p = x;
}
```

Errores con Punteros

- *En este caso también tenemos un error ¿Cuál es?*

```
int main( void )
{
  int x, *p;
  x = 10;
  p = x;
  printf( "El valor de x es: %d\n", *p );
}
```

Errores con Punteros

- *También se puede cometer errores al hacer una comparación.*

```
char c[80], y[80];
char p1, p2;

p1 = c;
p2 = y;
if( p1 < p2 )...
```

- *Esto asume que en memoria **y** se almacena después de **c**.*

¿Cuál es el error?

```
#include <string.h>
#include <stdio.h>

int main( void )
{
   char *p1;
   char c[80];
   p1 = c;
   do {
     gets( c );
     while( *p1 ) printf( "%c", *p1++ );
     printf( "\n" );
   } while( strcmp(c, "fin") );
}
```

Punteros y Arreglos

- *Los punteros y los arreglos tienen una estrecha relación.*
- *El nombre de un arreglo sin subíndice, significa un puntero a la primera posición del arreglo*
- *Generalmente los arreglos son recorridos más rápido con punteros que con subíndices.*

Punteros y Arreglos

- *En este caso:*

```
float a[10], *b;
b = a;
printf(”Segunda posición: %c\n”, b[1]);
```

Es una construcción perfectamente lícita y muestra la afirmación anterior, esto es:

```
float a[10], *b;
b = &a[0];
```

***b** apunta al primer elemento del arreglo.*

Punteros y Arreglos

- *Es decir que en:* `float a[10];`
- *Se reserva el espacio en memoria para 10 elementos del tipo float.*
- *En cambio en:* `float *b;`
- *Se declara un apuntador a float que puede dirigirse al comienzo del arreglo y de aquí en más se puede utilizar como si fuese* ***a****, resultando más rápido para recorrer.*

Punteros y Arreglos

- *En:*

```
int *a, b[10];
a = b;
```

- *Para referirnos al tercer elemento lo podemos hacer de dos formas:*

```
b[2]    ó    *(a+2)
```

- *Esto muestra que teniendo apuntada la primera posición del vector, podemos indicar dos posiciones más adelante y derreferenciar el apuntador.*

Incrementos de Punteros

- *El incremento o decremento de un puntero puede involucrar a la dirección contenida en él o al valor que esta almacena.*
- *Si queremos desplazar el puntero a la dirección siguiente hacemos:*

```
a++ ;
```

- *Si queremos incrementar el contenido de la variable almacenada en esa dirección.*

```
(*b)++ ;
```

```
#include <stdio.h>
int main( void ) {
  int *a, *b, c[4] = { 18,20,22,24 };
  a = b = c;
  printf("Visto por a es: %d\n", *a);
  printf("Visto por b es: %d\n", *b);
  printf( "\nAhora incrementamos los
           punteros\n\n" );
  a++ ; (*b)++ ;
  printf("Visto por a es: %d\n", *a);
  printf("Visto por b es: %d\n", *b);
}
```

Incrementos de Punteros

- *Salida*

```
Visto por a es: 18
Visto por b es: 18

Ahora incrementamos los punteros

Visto por a es: 20
Visto por b es: 19
```

Punteros y Arreglos

- *Dos formas de escribir la función puts().*

```
void puts( char frase[] ) {
  register int t;
  for( t=0; frase[t]; t++ )
    putchar( frase[t] );
  putchar( '\n' );
}

void puts( char *frase ) {
  while( *frase ) putchar( *frase++ );
  putchar( '\n' );
}
```

Punteros y Arreglos

- *Cuando pasamos un vector a una función, la dirección adonde apunta la estamos pasando por valor.*
- *Por este motivo en el programa anterior la función no modifica en forma permanente la dirección adonde apunta el parámetro frase.*
- *En el programa siguiente la función de impresión recibe un puntero a puntero.*

```
void imprimestr( char **frase );

int main( void ) {
  char fr[] = "Frase de prueba", *punt ;
  punt = fr;
  printf( "Direc. puntero: %p\n", punt );
  imprimestr( &punt );
  printf( "Direc. puntero: %p\n", punt );
  imprimestr( &punt );
  printf( "Direc. puntero: %p\n", punt );
}

void imprimestr( char **frase ) {
  int i;
  printf( "    CONTENIDO\n" );
  for(i=0;i<20;i++) putchar( **(frase)++ );
  putchar( '\n' );
}
```

Punteros y Arreglos

- *En este caso estamos en condiciones de pasar la dirección de un puntero por referencia (esto lo permite el puntero a puntero)*
- *El programa anterior tiene la salida:*

```
Direc. puntero: 0xbffffcd0
    CONTENIDO
Segmentation fault (core dumped)
```

¿Porqué?

Punteros y Arreglos

- *En cambio sin en la función imprimestr() reemplazamos la línea:*

```
for(i=0;i<20;i++) putchar( **(frase)++ );
```

por

```
for(i=0;i<20;i++) putchar( *(*frase)++ );
```

Tendremos la siguiente salida:

Punteros y Arreglos

```
Direc. puntero: 0xbffffcd0
    CONTENIDO
Frase de pruebaôüÿ¿
Direc. puntero: 0xbffffce4
    CONTENIDO
 „  ýÿ¿ ýÿ¿
Direc. puntero: 0xbffffcf8
```

¿Puede explicar el comportamiento?

Punteros y Arreglos

- *Por último si reemplazamos en la función imprimestr(), la línea:*

```
for(i=0;i<20;i++) putchar( *(*frase)++ );
```

por

```
for(i=0;i<20;i++) putchar( (**frase)++ );
```

Tendremos la siguiente salida:

Punteros y Arreglos

```
Direc. puntero: 0xbffffcd0
    CONTENIDO
FGHIJKLMNOPQRSTUVWXY
Direc. puntero: 0xbffffcd0
    CONTENIDO
Z[\]^_`abcdefghijklm
Direc. puntero: 0xbffffcd0
```

¿Porqué no se imprime la frase?

¿Qué sucede en la función que llamó?

Arreglos de Punteros

- *De la misma forma que podemos generar arreglos de cualquier tipo de elemento, también lo podemos hacer de punteros.*

```
int *z[20];
```

- ***z** es un arreglo de 20 posiciones. Cada una de ellas apunta a una dirección de número entero.*
- *Dicho de otra forma, estamos guardando las direcciones de 20 variables enteras.*

Arreglos de Punteros

- *Si queremos que el quinto componente del arreglo de punteros, apunte a la dirección de la variable **nomb**, debemos hacer lo siguiente:*

```
z[4] = &nomb
```

- *El contenido de esa dirección es:*

```
*z[4]
```

Arreglos de Punteros

- *El pasaje de un arreglo de punteros a una función se hace de la siguiente forma:*

```
void impresion( int *vec[] )
{
  char count;
  for( count=0; count<LIM; count++ )
    printf( "La dirección de %2d es:
    %p y su contenido es: %d\n",
    count, vec[count], *vec[count] );
}
```

Indirección Múltiple

- *Es posible declarar punteros a punteros lo que se hace de la forma siguiente:*

```
int a, b, c, *d, *e, *f, **g;
```

- ***g** apunta a un puntero a enteros. O sea que **g** podrá apuntar a d, e o f.*
- *La primera derreferenciación de **g** nos da una dirección de puntero a entero. La segunda nos da el contenido de la dirección.*

Notación Avanzada

- *Extremando la notación de punteros y en relación con arreglos, tenemos que para recorrer **val[2][3]***

```
*val[0]                 val[0][0]
*(val[0] + 1)           val[0][1]
*(val[0] + 2)           val[0][2]
* val[1]                val[1][0]
*(val[1] + 1)           val[1][1]
*(val[1] + 2)           val[1][2]
```

Notación Avanzada

- *Lo anterior es equivalente a:*

```
*(*val)                 val[0][0]
*(*val + 1)             val[0][1]
*(*val + 2)             val[0][2]
*(*(val+1))             val[1][0]
*(*(val+1) + 1)         val[1][1]
*(*(val+1) + 2)         val[1][2]
```